

Тест 2

Функционално програмиране C#

1. Какво представлява Func<int, int>?

- A) Делегат без вход и без изход
- B) Делегат с вход **int** и изход **int**
- C) Делегат с два входа **int**
- D) Процедура без връщана стойност

1. Какво представлява Func<int, int>?

- A) Делегат без вход и без изход
- B) Делегат с вход **int** и изход **int****
- C) Делегат с два входа **int**
- D) Процедура без връщана стойност

Отговор:

Func<int,int> приема аргумент от тип **int** и връща резултат от тип **int**.
Последният параметър винаги е типът на резултата.

2. Кое от следните е вярно за `Action<T>`?

- A) Връща стойност
- B) Връща `bool`
- C) Не връща стойност
- D) Работи само с `int`

2. Кое от следните е вярно за Action<T>?

- A) Връща стойност
- B) Връща bool
- C) Не връща стойност
- D) Работи само с int

Отговор:

Action<T> не връща стойност (void).

Използва се за операции със странични ефекти.

3. Какъв е типът на ламбдата $x \Rightarrow x.Length$?

- A) `Func<int, int>`
- B) `Func<string, int>`
- C) `Action<string>`
- D) `Predicate<string>`

3. Какъв е типът на ламбдата $x \Rightarrow x.Length$?

- A) `Func<int, int>`
- B) `Func<string, int>`
- C) `Action<string>`
- D) `Predicate<string>`

Отговор:

`x.Length` връща `int`, а `x` е `string`. Следователно типът е `Func<string,int>`.

4. Кое твърдение за Predicate<T> е вярно?

- A) Връща T
- B) Връща int
- C) Връща bool
- D) Не приема параметри

4. Кое твърдение за Predicate<T> е вярно?

- A) Връща T
- B) Връща int
- C) Връща bool
- D) Не приема параметри

Отговор:

Predicate<T> винаги връща bool. Използва се за проверки и условия.

Predicate<T> е просто Func<T, bool>, но със специално име за четимост.

5. Какъв ще е резултатът от:

```
Predicate<int> isEven = x => x % 2 == 0;  
isEven(3);
```

A) true

B) false

C) 3

D) Грешка при компиляция

5. Какъв ще е резултатът от:

```
Predicate<int> isEven = x => x % 2 == 0;  
isEven(3);
```

A) true

B) false

C) 3

D) Грешка при компилация

Отговор: Числото 3 е нечетно, затова $x \% 2 == 0$ връща false.

6. Кое от следните е страничен ефект?

```
Action<int> addToList = x => list.Add(x);
```

- A) Ламбда-изразът
- B) Делегатът
- C) Промяната на списъка
- D) Типът Action

6. Кое от следните е страничен ефект?

```
Action<int> addToList = x => list.Add(x);
```

A) Ламбда-изразът

B) Делегатът

C) Промяната на списъка

D) Типът Action

Отговор:

`list.Add(x)` променя външен обект (списъка).

Това е класически страничен ефект.

7. Какво позволява комбинирането hello + bye?

```
Action<string> hello = name => Console.WriteLine("Hello " + name);  
Action<string> bye = name => Console.WriteLine("Goodbye " + name);  
  
Action<string> both = hello + bye;  
both("Ivan");
```

- A) Унаследяване
- B) Полиморфизъм
- C) Multicast делегат
- D) Generics

7. Какво позволява комбинирането hello + bye?

```
Action<string> hello = name => Console.WriteLine("Hello " + name);  
Action<string> bye = name => Console.WriteLine("Goodbye " + name);  
  
Action<string> both = hello + bye;  
both("Ivan");
```

- A) Унаследяване
- B) Полиморфизъм
- C) Multicast делегат
- D) Generics

Отговор:

При извикване `Action<string> both = hello + bye` се изпълняват всички функции по ред.

8. Какъв е типът на `Func<int>`?

- A) `int → int`
- B) `() → int`
- C) `int → void`
- D) `() → void`

8. Какъв е типът на `Func<int>`?

A) `int → int`

B) `() → int`

C) `int → void`

D) `() → void`

Отговор:

`Func<int>` означава функция без параметри, която връща `int`. Това е еквивалент на `() → int`.

9. Кое от следните НЕ е вярно за ламбда-изрази?

- A) Могат да се присвояват на делегати
- B) Могат да имат странични ефекти
- C) Задължително са чисти функции
- D) Могат да връщат стойност

9. Кое от следните НЕ е вярно за ламбда-изрази?

- A) Могат да се присвояват на делегати
- B) Могат да имат странични ефекти
- C) Задължително са чисти функции
- D) Могат да връщат стойност

Отговор:

Ламбда-изразите не са задължително чисти. Те могат да имат странични ефекти.

10. Какъв е резултатът от:

```
Func<int, int> f = x => x * 2;  
f(2);
```

A) 2

B) 4

C) 8

D) Грешка

10. Какъв е резултатът от:

```
Func<int, int> f = x => x * 2;  
f(2);
```

A) 2

B) 4

C) 8

D) Грешка

Отговор:

Функцията умножава входа по 2. При вход 2 резултатът е 4.

11. Какво означава Compose(f, g)?

- A) $g(f(x))$
- B) $f(g(x))$
- C) $f(x) + g(x)$
- D) $f(x, g(x))$

```
static Func<T3, T2> Compose<T1, T2, T3>  
    ( Func<T1, T2> f, Func<T3, T1> g)
```

11. Какво означава Compose(f, g)?

A) $g(f(x))$

B) $f(g(x))$

C) $f(x) + g(x)$

D) $f(x, g(x))$

```
static Func<T3, T2> Compose<T1, T2, T3>  
    ( Func<T1, T2> f, Func<T3, T1> g)
```

Отговор:

По дефиниция $(f \circ g)(x) = f(g(x))$. Първо се изпълнява g , после f .

12. Какъв е типът на Compose(ToDouble, ParseInt)?

A) A) $\text{int} \rightarrow \text{double}$

B) $\text{string} \rightarrow \text{int}$

C) $\text{string} \rightarrow \text{double}$

D) $\text{double} \rightarrow \text{string}$

```
Func<string, int> ParseInt = s => int.Parse(s);
```

```
Func<int, double> ToDouble = i => (double)i;
```

```
static Func<T3, T2> Compose<T1, T2, T3>  
    ( Func<T1, T2> f, Func<T3, T1> g)  
    => x => f(g(x));
```

12. Какъв е типът на Compose(ToDouble, ParseInt)?

A) A) $\text{int} \rightarrow \text{double}$

B) $\text{string} \rightarrow \text{int}$

C) $\text{string} \rightarrow \text{double}$

D) $\text{double} \rightarrow \text{string}$

```
Func<string, int> ParseInt = s => int.Parse(s);
```

```
Func<int, double> ToDouble = i => (double)i;
```

```
static Func<T3, T2> Compose<T1, T2, T3>  
    ( Func<T1, T2> f, Func<T3, T1> g )  
    => x => f(g(x));
```

Отговор:

ParseInt е $\text{string} \rightarrow \text{int}$, ToDouble е $\text{int} \rightarrow \text{double}$. Композицията дава $\text{string} \rightarrow \text{double}$.

13. Кое е вярно за композицията?

- A) Редът няма значение
- B) Работи само за еднакви типове
- C) Изходът на едната е вход за другата
- D) Използва цикли

13. Кое е вярно за композицията?

- A) Редът няма значение
- B) Работи само за еднакви типове
- C) Изходът на едната е вход за другата
- D) Използва цикли

Отговор:

Изходът на едната функция трябва да съвпада с входа на другата.
Това е основното правило на композицията.

14. Какъв ще е резултатът от:

`Compose(AddVAT, ToDouble)(100)`

- A) 100
- B) 120
- C) 1.2
- D) Грешка

`Func<int, double> ToDouble = i => (double)i;`

`Func<double, double> AddVAT = d => d * 1.20;`

14. Какъв ще е резултатът от:

`Compose(AddVAT, ToDouble)(100)`

A) 100

B) 120

C) 1.2

D) Грешка

Отговор:

`ToDouble(100)` приема 100 и връща 100.0. `AddVAT` умножава 100.0 по 1.2 и връща резултат 120.

15. Какво е предимство на композицията?

- A) Повече междинни променливи
- B) По-малко функции
- C) Pipeline без междинни стойности
- D) По-бързо изпълнение винаги

15. Какво е предимство на композицията?

- A) Повече междинни променливи
- B) По-малко функции
- C) Pipeline без междинни стойности
- D) По-бързо изпълнение винаги

Отговор:

Композицията позволява pipeline без междинни променливи.
Това е ключов принцип във функционалния стил.

16. Кое твърдение за `Compose<T1,T2,T3>` е вярно?

- A) Работи само с числа
- B) Работи само с `int`
- C) Позволява различни входни/изходни типове
- D) Не е generic

16. Кое твърдение за Compose<T1,T2,T3> е вярно?

- A) Работи само с числа
- B) Работи само с int
- C) Позволява различни входни/изходни типове
- D) Не е generic

Отговор:

Generic версията позволява различни входни и изходни типове.
Това я прави много гъвкава.

17. Кое НЕ е вярно за композицията?

- A) Намалява нуждата от променливи
- B) Подобрява четимостта
- C) Заменя рекурсията
- D) Работи с делегати

17. Кое НЕ е вярно за композицията?

- A) Намалява нуждата от променливи
- B) Подобрява четимостта
- C) Заменя рекурсията
- D) Работи с делегати

Отговор:

Композицията **не замества рекурсията**.

Тя комбинира функции, а не управлява повторение.

18. Коя функция е чиста?

A) `Console.WriteLine(x)`

B) `x => x + 1`

C) `list.Add(x)`

D) `counter++`

18. Коя функция е чиста?

A) Console.WriteLine(x)

B) $x \Rightarrow x + 1$

C) list.Add(x)

D) counter++

Отговор:

$x \Rightarrow x + 1$ е чиста функция, защото:

- не променя външно състояние;
- няма странични ефекти;
- винаги връща един и същ резултат при едни и същи аргументи;

19. Защо MultyNum<T> използва INumber<T>?
static T MultyNum<T> where T : INumber<T>

- A) За сравнение
- B) За цикли
- C) За аритметични операции
- D) За стрингове

19. Защо MultyNum<T> използва INumber<T>?
static T MultyNum<T> where T : INumber<T>

- A) За сравнение
- B) За цикли
- C) За аритметични операции
- D) За стрингове

Отговор:

INumber<T> позволява използване на аритметични оператори. Без него * не е разрешен за generic тип.

20. Какво гарантира constraint
where $T : IComparable<T>$?

- A) Събиране
- B) Умножение
- C) Сравнение
- D) Деление

20. Какво гарантира constraint where T :
IComparable<T>?

- A) Събиране
- B) Умножение
- C) Сравнение
- D) Деление

Отговор:

IComparable<T> гарантира, че стойностите могат да се сравняват.
Това е нужно за тях логика.

21. Кое е вярно за generics?

- A) Работят само с числа
- B) Заменят делегатите
- C) Позволяват повторна употреба на код
- D) Са по-бавни винаги

21. Кое е вярно за generics?

- A) Работят само с числа
- B) Заменят делегатите
- C) Позволяват повторна употреба на код
- D) Са по-бавни винаги

Отговор:

Generics позволяват един и същ код да работи с различни типове. Това увеличава повторната употреба.

22. Какво е базов случай при рекурсия?

- A) Първият елемент
- B) Последният елемент
- C) Условие за спиране
- D) Цикъл

22. Какво е базов случай при рекурсия?

- A) Първият елемент
- B) Последният елемент
- C) Условие за спиране
- D) Цикъл

Отговор:

Базовият случай спира рекурсията. Без него се получава безкрайно извикване.

23. Какво означава head в списък?

- A) Последният елемент
- B) Първият елемент
- C) Целият списък
- D) Празният списък

23. Какво означава head в списък?

- A) Последният елемент
- B) Първият елемент**
- C) Целият списък
- D) Празният списък

Отговор:

head е първият елемент на списъка. С него се работи във всяка рекурсивна стъпка.

24. Какво представлява tail?

- A) Първият елемент
- B) Последният елемент
- C) Списък без първия елемент
- D) Празен списък

24. Какво представлява tail?

- A) Първият елемент
- B) Последният елемент
- C) Списък без първия елемент
- D) Празен списък

Отговор:

tail е списъкът без първия елемент.

25. Какво е предимство на опашковата рекурсия?

- A) По-лесен код
- B) По-малко памет
- C) Повече странични ефекти
- D) Повече променливи

25. Какво е предимство на опашковата рекурсия?

- A) По-лесен код
- B) По-малко памет
- C) Повече странични ефекти
- D) Повече променливи

Отговор:

Опашковата рекурсия може да се оптимизира и използва по-малко стек памет. Това е основното ѝ предимство